Master 2 Internship report

Dynamical properties of the Hopfield model through bitwise arithmetic

Bastien Dumont

Supervised by Michele Castellana and David Lacoste

Institut Curie and ESPCI, Paris

March 23 - July 17, 2026

Master de Physique fondamentale et applications, parcours Physique

Scholar year 2025 - 2026

July 3, 2026

placeholder

Abstract

placeholder

Contents

1 Introduction

1.1 Presentation of the host laboratories

This internship was conducted as part of my Master 2 at the Magistère of Fundamental Physics of Paris-Saclay Université, which I completed at Aix-Marseille Université in the "Physique Fondamentale et Applications" track. It was carried out at the "Physique des Cellules et Cancer" (PCC, UMR 168) laboratory of the Institut Curie as well as at the Gulliver laboratory (UMR 7083) of the "École Supérieure de Physique et de Chimie Industrielles" (ESPCI) in Paris, and was supervised by Michele Castellana (Institut Curie) and David Lacoste (ESPCI). Although the PCC unit is a leading biological physics laboratory, part of the Institut Curie – a pioneer institution in cancer research and treatment – it hosts a renowned theory team (of which Michele Castellana is a member) that is sometimes interested in physics-driven topics such as the one investigated in this report. The Gulliver laboratory hosts both experimentalists and theoreticians (among whom David Lacoste), working at the frontier between physical, chemical and biological topics.

1.2 Subject

In October 2024, John J. Hopfield was awarded the Nobel Prize in Physics alongside Geoffrey Hinton for their pioneering work on artificial neural networks [1]. Their work laid the foundations for further development and understanding of neural networks, whether they are organic or artificial, and eventually gave rise to Artificial Intelligence (AI).
Hopfield worked on an artificial neural network, originally invented by Shun'ichi Amari in 1972 [2]. He greatly contributed to the analysis, understanding and popularization of the network in his landmark paper *Neural networks and physical systems with emergent collective computational abilities* published in 1982 [3], and the network was eventually named after him. The model, described in more details in subsection 5.1, is similar to the Ising and the Spin Glass networks: it is composed of a set of spins – called neurons in the case of the Hopfield network $-S_i \in \{-1, 1\}$ interacting with one another. In the same way as the two preceding models, the Hopfield model is energy-based, that is it will tend to minimize its energy over time. What makes this model different from the other two is the way the interactions between neurons are defined: as for the Ising and Spin Glass models they remain static over time but depend on the set of patterns to be stored in the network, making the latter attractors – minima of energy – of the model.
The Hopfield model provided the first simple yet powerful framework for understanding networks of interconnected neurons. It has become a reference in statistical physics on networks and its properties at equilibrium have been extensively studied and are now well known, whether it is its statistical mechanics [4, 5, 6] or its phase diagram and transitions [7]. However, much less is known about the dynamics that lead to those well-known equilibrium properties.

1.3 Objectives

The main objective of this internship is to efficiently simulate the Metropolis dynamics on the Hopfield network. To this end, we use a bitwise formalism that allows us to simulate several replicas of the system in parallel, increasing the efficiency of the exploration of the model's energy landscape. Our objectives in this internship are thus twofold: first, to validate the bitwise implementation and quantify its computational gain in yield over the standard Metropolis algorithm; second, to leverage this implementation to study the dynamics of the Hopfield network. To validate the implementation, we begin with the Ising model, which serves as a natural benchmark due to its simplicity and well-characterized phase transition. Once this implementation has been proven sane and efficient on the Ising network, we increase the complexity by implementing it on the Spin Glass network, testing the gain in yield again.
Once the bitwise arithmetic has been validated on both models, we implement it on the Hopfield network in order to analyze its dynamics.

2 Implementation of the algorithm

We start by presenting the broad implementation of the Metropolis algorithm, before showing how we adapt it to each specific model.

2.1 The Metropolis algorithm

In order to make the models evolve in time, which is necessary to observe their dynamics, we use the well-known metropolis algorithm [8]. For models with a defined Hamiltonian, the Metropolis algorithm proceeds as follows:

1. Start from an initial configuration $\{S_i\}$.
2. Repeat N times:
   a. Pick a random spin S_k and compute the energy change ΔE_k upon flipping it.
   b. If $\Delta E_k \leq 0$, accept the flip.
   c. Else, draw a random number $r \sim \mathcal{U}([0,1])$ and accept the flip if $r \leq e^{-\beta \Delta E_k}$.
3. Repeat Step 2 for N_{sweeps} sweeps.

Where N is the number of spins, β is the inverse temperature, and N_{sweeps} is the number of sweeps, where one sweep consists of N successive single-spin flip attempts.
This algorithm however becomes computationally demanding when one attempts to sample a large number of configurations. This situation occurs, for example, when looking for rare samples in the dynamics, or sampling many different initial conditions – both of which we will try to explore during this internship. We thus need to find way of optimizing the implementation of the algorithm.

2.2 First optimization: the bitwise arithmetic

The first optimization in the implementation of the Metropolis algorithm is the use of bitwise arithmetic. The main idea is to leverage the boolean representation as stored in our computers. The typical object that we manipulate is called `Bits`: it is a collection of $n_{\text{bits}} = 64$ boolean variables. In that way, we have the possibility to simulate 64 evolutions of the same variable in parallel.

$$W = \underbrace{\boxed{b^0}\boxed{b^1}\boxed{b^2}\cdots\boxed{b^{62}}\boxed{b^{63}}}_{64 \text{ bits} \leftrightarrow 64 \text{ replicas}}$$

Figure 1: A `Bits` W encoding 64 parallel replicas. Each bit b_i carries the state of replica i.

This structure is very convenient to represent binary variable such as the spins of the system. The boolean representation $b_i \in \{0, 1\}$ of a spin $s_i \in \{-1, +1\}$ is given by $b_i = (s_i + 1)/2$. In that way, we are able to implement several replicas of the full spin system in a set of `Bits` $\{S_i\}$, each encoding the replicas of the associated spin, as shown in Figure 2.

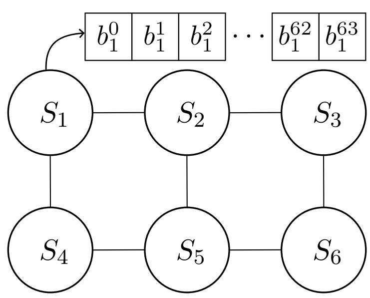

Figure 2: Bitwise implementation of a spin system. Each replica of the system is obtained by selecting the corresponding replica in each spin.

Storing all replicas in a single `Bits` object allows us to manipulate and update them simultaneously using a single bitwise operator. This is an instance of SWAR (SIMD Within A Register) parallelism, where SIMD stands for Single Instruction, Multiple Data: all 64 replicas are updated by a single bitwise instruction. The restriction to $n_{\text{bits}} = 64$ reflects the native word size of modern 64-bit CPUs [9]. This could in principle be extended to 512 bits using AVX-512 SIMD instructions, at the cost of introducing a new register type in the implementation. This will be left for further discussion.
The convention used for what follows is presented in the Table 1:

Object	Notation
Scalar spin	$s_i \in \{-1, +1\}$
Scalar bit	$b_i \in \{0, 1\}$
64 replicas of spin i	$S_i \in \{-1, 1\}^{64}$
64-bit word (`Bits`)	$B_i \in \{0, 1\}^{64}$
Replica k of spin i	$s_i^{(k)} \in \{-1, +1\}$
Replica k of bit i	$b_i^{(k)} \in \{0, 1\}$

Table 1: Notation for the bitwise representation of spins.

Now that we know how to conveniently handle binary variables such as spins, we need to employ the same formalism to represent integer, such as the sum ΔE_k. This is done using a `BitSet`, which is a collection of `Bits`. When each `Bits` is associated with a power of 2, the `BitSet` represent a collection of n_{bits} unsigned integers and the `BitSet` is called `UnsignedInt`. We show an example of `UnsignedInt` in the Figure 3.

$$
Z = \begin{array}{c}
\underbrace{\begin{array}{cccccc}
\boxed{b_0^0} & \boxed{b_0^1} & \boxed{b_0^2} & \cdots & \boxed{b_0^{62}} & \boxed{b_0^{63}} \\
\boxed{b_1^0} & \boxed{b_1^1} & \boxed{b_1^2} & \cdots & \boxed{b_1^{62}} & \boxed{b_1^{63}} \\
 & \vdots & & & & \\
\boxed{b_l^0} & \boxed{b_l^1} & \boxed{b_l^2} & \cdots & \boxed{b_l^{62}} & \boxed{b_l^{63}}
\end{array}}_{\text{64 replicas}}
\end{array}
\begin{array}{c} 2^0 \\ 2^1 \\ \vdots \\ 2^l \end{array}
$$

Figure 3: An `UnsignedInt` encoding 64 parallel unsigned integers. Each row is a `Bits` associated with a power of 2. The value of each integer is obtained by reading the `UnsignedInt` along the corresponding column. For instance, the fourth integer stored in Z is $Z_4 = \sum_{i=0}^{l} b_4^i \, 2^i$.

In the same way as for `Bits`, the `UnsignedInt` data structure enables us to add, subtract or even multiply two sets of 64 integers with just the cost of one operation.
This implementation method has proven efficient when dealing with discreet values models such as the Ising [10] or the Spin Glass [11] models, but it has not yet been employed to study the Hopfield network.

2.3 Second Optimization: Shared random numbers

The bitwise implementation is a structural optimization of the data representation. On top of this structural optimization can be added a physical one which consists in having all replicas share the same sequence of random numbers, fully parallelizing their evolution. This turns out to be central: random number generation is often the simulation bottleneck, as it involves costly operations such as multiplications and is called at every step of the hot loop. A schematic view of the shared random number is shown in Figure 4. Using the same random number among all evolutions raises the questions of the independence of the realizations, this will be the subject of the next subsection.

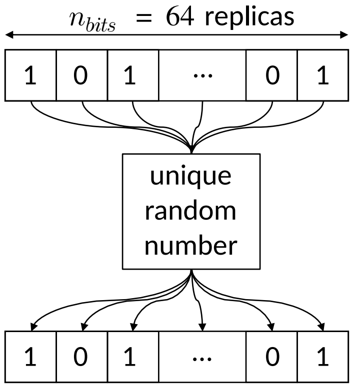

Figure 4: All 64 replicas of the system share the same random number at each step.TO BE CHANGED (BOTTOW ROW)

2.4 Note on the independence of the realizations

2.5 Structure of the code

The simulation of the dynamics has been entirely coded in `C++`, using an object-oriented approach to the problem.

$$S_1 = \boxed{b_1^0}\boxed{b_1^1}\boxed{b_1^2}\cdots\boxed{b_1^{62}}\boxed{b_1^{63}}$$

3 The Ising model

We first implement the method on the Ising model, which is the simplest one.

3.1 The model

The Hamiltonian of the Ising network is given by

$$H_{\text{Ising}} = -J \sum_{<i,j>} S_i S_j - h \sum_i S_i,$$

which, in absence of external field h becomes,

$$H_{\text{Ising},\ h=0} = -J \sum_{<i,j>} S_i S_j. \tag{1}$$

When flipping the spin S_k ($S_k \to -S_k$), the associated energy shift is

$$\begin{aligned} \Delta E_k &= H_{\text{Ising}}\left[-S_k\right] - H_{\text{Ising}}\left[S_k\right] \\ &= 2JS_k \sum_{i\in\partial k} S_i. \end{aligned} \tag{2}$$

where $i \in \partial k$ denotes the summation over the neighbors of k. If $\Delta E_k \leq 0$ the flip is accepted. Else, it is accepted if

$$\begin{aligned} & r \leq e^{-\beta \Delta E_k} \\ \implies & \frac{-1}{\beta} \log r \geq \Delta E_k \\ \implies & \frac{-1}{2\beta J} \log r \geq S_k \sum_{i\in\partial k} S_i. \end{aligned} \tag{3}$$

With $r \sim \mathcal{U}([0,1])$, we know that $-\log r \sim \mathcal{E}(1)$ [] (source?), thus $-(2\beta J)^{-1} \log r = \rho \sim \mathcal{E}(2\beta J)$. The left-hand side is a random variable of probability density $p(\rho) = 2\beta J e^{-2\beta J}$ and mean $(2\beta J)^{-1}$. The right-hand side of the equation is an integer which can be positive of negative. However, due to the first escape condition of the algorithm, we know that it is strictly positive. The left-hand side is thus compared with a positive integer. We can thus only consider the whole part of it without changing the inequality in order to obtain an integer-only flip condition:

$$\lfloor \rho \rfloor \geq S_k \sum_{i\in\partial k} S_i. \tag{4}$$

We now need to write the sum on spins in a `Bits` formalism in order to properly parallelize the evolutions.

3.2 Bitwise Implementation

Remembering that S_i is actually an array containing 64 replicas of the same spin, we write $S_i = -1+2B_i$, with B_i a `Bits`. The sum hence becomes:

$$\begin{aligned} \sum_{i\in\partial k} S_k S_i &= \sum_{i\in\partial k} (-1+2B_k)(-1+2B_i) \\ &= \sum_{i\in\partial k} 1 - 2(B_k + B_i) + 4B_k.B_i, \end{aligned} \tag{5}$$

where n_k is the number of neighbors of the spin k. We show in Table 2 that the term in the sum can be re-written using the XNOR ($\odot$) logical operator. Thus, the term in the sum is simply equal to $2B_k \odot B_i - 1$, where $\odot$ denotes the bitwise XNOR operation applied to the corresponding bits of each `Bits`. The sum thus becomes:

$$\sum_{i\in\partial k} S_k S_i = 2 \sum_{i\in\partial k} C_{ki} - n_k, \tag{6}$$

b_k	b_i	$1 - 2(b_k + b_i) + 4b_k.b_i$	$b_k \odot b_i$	$2b_k \odot b_i - 1$
0	0	1	1	1
0	1	-1	0	-1
1	0	-1	0	-1
1	1	1	1	1

Table 2: The summed term can be simplified using a XNOR operator.

with $C_{ki} = 2B_k \odot B_i$ and n_k the number of neighbor of the spin k. The bitwise flip condition is then:

$$\lfloor \rho \rfloor + n_k \geq 2 \sum_{i \in \partial k} C_{ki}. \tag{7}$$

In addition, according to the definition of ΔE_k (2), we have $-2Jn_k \leq \Delta E_k \leq 2Jn_k$. Plugging this into Eq (3) yields the flip escape condition:

$$\lfloor \rho \rfloor \geq n_k. \tag{8}$$

When satisfied, this guarantees that Eq. (4) holds for every replica regardless of its local configuration, so that all replicas are flipped simultaneously without computing the per-replica bitwise test. This first escape is useful, as it avoids the bitwise flipping test calculations for each replica, even when done in parallel. Instead, a simple integer comparison suffices. This condition is typically met in the high-temperature regime. Moreover, the integer flip condition (4) constrains the range of values taken by $\lfloor \rho \rfloor$ and $2\sum_{i \in \partial k} C_{ki}$ to n_k and $2n_k$, respectively. This is particularly convenient, as it allows the `UnsignedInt` objects to be allocated with static sizes at initialization, removing the need for dynamic resizing during the simulation.
Knowing that, we can write the pseudocode for the evolution of the Ising network under the Metropolis algorithm:

Algorithm 1: Ising Metropolis sweep (bitwise)

Input: Lattice spins $\{S_i\}$, couplings J, inverse temperature β
Output: Updated lattice spins $\{S_i\}$

```
Bits xnor, mask
UnsignedInt T_k, threshold
for sweep = 1 to N_sweeps do
    for step = 1 to N do
        Draw ⌊ρ⌋ with ρ ~ E(2βJ)
        if ⌊ρ⌋ ≥ n_k then
            S_k ← −S_k        // escape condition: unconditional flip across all replicas
        else
            T_k ← 0
            threshold ← 0
            for i ∈ ∂k do
                xnor ← B_k ⊙ B_i                 // bitwise XNOR across replicas
                T_k += xnor                      // increment agreement count per replica
            T_k ← 2T_k
            threshold ← ⌊ρ⌋ + n_k
            mask ← (T_k ≤ threshold)             // per-replica flip condition
            B_k ← B_k ⊕ mask                     // bitwise spin flip: only replicas
                                                 // for which mask^(ℓ) = 1 are flipped
```

3.3 Phase diagram

3.4 Performance test

4 The Spinglass model

4.1 The model

4.2 Bitwise Implementation

4.3 Performance test

5 The Hopfield model

5.1 The model

5.2 Bitwise Implementation

5.2.1 Naive implementation

5.2.2 Optimized implementation

5.3 Capacity of the Hopfield network

5.4 Performance test

6 Conclusion

References

[1] The Royal Swedish Academy of Sciences. Press release: The Nobel Prize in Physics 2024. `https://www.nobelprize.org/prizes/physics/2024/press-release/`, October 2024. Accessed: June 2026.

[2] Shun-ichi Amari. Learning patterns and pattern sequences by self-organizing nets of threshold elements. *IEEE Transactions on Computers*, C-21:1197–1206, 1972.

[3] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.

[4] Daniel J. Amit, Hanoch Gutfreund, and H. Sompolinsky. Spin-glass models of neural networks. *Phys. Rev. A*, 32:1007–1018, Aug 1985.

[5] Daniel J Amit, Hanoch Gutfreund, and H Sompolinsky. Statistical mechanics of neural networks near saturation. *Annals of Physics*, 173(1):30–67, 1987.

[6] Daniel J. Amit. *Modeling Brain Function: The World of Attractor Neural Networks*. Cambridge University Press, Cambridge, 1989.

[7] John Hertz, Anders Krogh, and Richard G. Palmer. *Introduction to the Theory of Neural Computation*. Santa Fe Institute Studies in the Sciences of Complexity. Addison-Wesley, Redwood City, CA, 1991.

[8] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[9] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual*, 2023.

[10] H Freund and P Grassberger. Multispin coding for spin glasses. *Journal of Physics A: Mathematical and General*, 21(16):L801, aug 1988.

[11] Matteo Palassini and Sergio Caracciolo. Monte carlo simulation of the three-dimensional ising spin glass, 1999.